

Gerenciador de Tabela de Símbolos usando Pilha de Tabelas
Hash

Maria Layanne Rodrigues da Silva

Maceió -AL 2026

1 Introduction

Os compiladores são ferramentas fundamentais no desenvolvimento de software, sendo responsáveis por traduzir programas escritos em linguagens de alto nível para uma linguagem compreensível pelo computador [1]. Durante esse processo, diversas estruturas e mecanismos são utilizados para armazenar e organizar informações necessárias à análise e validação do código-fonte.

Entre esses mecanismos destaca-se a Tabela de Símbolos, estrutura responsável por registrar informações sobre identificadores, como nomes de variáveis e seus respectivos tipos, permitindo que o compilador realize verificações semânticas e controle adequadamente os diferentes escopos de um programa [1].

O presente trabalho tem como objetivo implementar um Gerenciador de Tabela de Símbolos utilizando uma pilha de tabelas hash, simulando o comportamento empregado por compiladores durante o gerenciamento de escopos aninhados. A aplicação permite criar e remover escopos, declarar variáveis e realizar buscas respeitando as regras de visibilidade definidas pelos diferentes níveis de escopo.

A implementação foi desenvolvida na linguagem Python utilizando estruturas de dados nativas, possibilitando a demonstração prática dos conceitos estudados na disciplina de Compiladores. Este projeto também compõe o processo avaliativo da disciplina, funcionando como uma atividade prática para consolidar os conhecimentos abordados em sala de aula [2].

2 Metodologia de Implementação

A implementação foi desenvolvida utilizando a linguagem Python [3] e fundamenta-se no conceito de pilha de tabelas hash para representar os diferentes escopos de um programa [4].

A estrutura principal do sistema consiste em uma lista Python (`list`), utilizada como uma pilha de escopos [4]. Cada escopo é representado por um dicionário (`dict`), estrutura baseada em tabela hash responsável por armazenar pares contendo o nome da variável e seu respectivo tipo [4].

Inicialmente, o sistema cria automaticamente um escopo global, que permanece ativo durante toda a execução do programa. Novos escopos podem ser adicionados sobre esse escopo global, formando uma pilha de escopos aninhados. Sempre que um novo escopo é criado, uma nova tabela hash é adicionada ao topo da pilha. Da mesma forma, quando um escopo é removido, a tabela hash correspondente é retirada da estrutura por meio da operação de desempilhamento [4].

A funcionalidade de declaração de variáveis foi implementada por meio do método `declarar(variavel, tipo)`, responsável por inserir novas entradas na tabela hash do escopo atual. Antes da inserção, o sistema verifica se a variável já foi declarada naquele mesmo escopo, evitando duplicidades.

A busca de variáveis é realizada pelo método `buscar(variavel)`, que percorre os escopos do mais interno para o mais externo. Esse comportamento reproduz o mecanismo utilizado pelos compiladores para localizar identificadores e respeitar as regras de visibilidade entre escopos aninhados [1].

Além das funcionalidades principais, foram implementadas rotinas de validação para nomes de variáveis e tipos permitidos, bem como um menu interativo em terminal para facilitar a execução e os testes da aplicação.

Dessa forma, a solução proposta demonstra na prática a utilização conjunta de pilhas e tabelas hash para o gerenciamento de símbolos [4], reproduzindo um dos componentes fundamentais presentes na fase de análise semântica de compiladores [1].

3 Casos de Teste

Os casos de teste foram realizados por meio da execução do programa no ambiente Google Colab. As evidências apresentadas nesta seção consistem em capturas de tela (*prints*) obtidas durante a execução do sistema, demonstrando as entradas fornecidas pelo usuário e as respectivas saídas produzidas pela aplicação. Esses registros foram utilizados para validar o funcionamento

das funcionalidades implementadas.

3.1 Caso de Teste 1 – Criação de Escopo e Declaração de Variável

- **Objetivo:** Verificar se o sistema cria um novo escopo e registra uma variável.
- **Entrada:** 1 - Criar escopo → 3 - Declarar variável (Variável: idade, Tipo: int).
- **Saída Esperada:** Novo escopo criado. / Declarada: idade -> int.

```
=== MENU ===
... 1 - Adicionar escopo
    2 - Remover do escopo
    3 - Declarar variável
    4 - Buscar variável
    5 - Mostrar tabela
    0 - Sair
Escolha: 1
Novo escopo criado.

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3

Exemplos de nomes válidos:
x
idade
nome_cliente
_total
valor123

Nome da variável: idade

Tipos disponíveis:
int
float
string
char
bool

Tipo da variável: int
Declarada: idade -> int
```

Figura 1: Evidência do Caso de Teste 1: Criação de escopo e declaração de variável.

3.2 Caso de Teste 2 – Busca de Variável Existente

- **Objetivo:** Verificar se a busca encontra corretamente uma variável previamente declarada.
- **Entrada:**
 - 4 - Buscar variável (Variável: idade)
- **Saída Esperada:**
Tipo encontrado: int

```
Tipo da variável: int
Declarada: idade -> int

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 4
Variável a buscar: idade
Tipo encontrado: int
```

Figura 2: Evidência do Caso de Teste 2: Busca de variável existente.

3.3 Caso de Teste 3 – Busca de Variável Inexistente

- **Objetivo:** Verificar o comportamento do sistema quando a variável não existe em nenhum escopo.
- **Entrada:**
 - 4 - Buscar variável (Variável: salario)
- **Saída Esperada:**
Variável não encontrada.

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 4
Variável a buscar: salario
Variável não encontrada.
```

Figura 3: Evidência do Caso de Teste 3: Busca de variável inexistente.

3.4 Caso de Teste 4 – Escopos Aninhados

- **Objetivo:** Verificar prioridade da variável no escopo mais interno.
- **Entrada:** 1 - Criar escopo → 3 - Declarar (x, int) → 1 - Criar escopo → 3 - Declarar (x, float) → 4 - Buscar (x).
- **Saída:** Novo escopo criado / Declarada: x -> int / Novo escopo criado / Declarada: x -> float / Tipo encontrado: float.

Saída Esperada:

```
Novo escopo criado.
Declarada: x -> int
Novo escopo criado.
Declarada: x -> float
Tipo encontrado: float
```

```

    ...  === MENU ===
          1 - Adicionar escopo
          2 - Remover do escopo
          3 - Declarar variável
          4 - Buscar variável
          5 - Mostrar tabela
          0 - Sair
    Escolha: 1
    Novo escopo criado.

    Escolha: 3
    Nome da variável: x
    Tipo da variável: int
    Declarada: x -> int

    Escolha: 1
    Novo escopo criado.

    Escolha: 3
    Nome da variável: x
    Tipo da variável: float
    Declarada: x -> float

    Escolha: 4
    Variável a buscar: x
    Tipo encontrado: float

```

Figura 4: Evidência do Caso de Teste 4: Regra de prioridade em escopos aninhados.

3.5 Caso de Teste 5 – Remoção de Escopo

- **Objetivo:** Verificar se a remoção do escopo restaura a visibilidade da variável declarada no escopo externo.
- **Entrada:**
 - 2 - Remover escopo
 - 4 - Buscar variável (Variável: x)
- **Saída Esperada:**
 - Escopo removido.
 - Tipo encontrado: int

```

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 2
Escopo removido.

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 4
Variável a buscar: x
Tipo encontrado: int

```

Figura 5: Evidência do Caso de Teste 5: Remoção de escopo e restauração da visibilidade externa.

3.6 Caso de Teste 6 – Redeclaração de Variável

- **Objetivo:** Verificar se o sistema impede a declaração duplicada de variáveis dentro do mesmo escopo.
- **Entrada:**
 - 3 - Declarar variável (Nome da variável: `idade`, Tipo da variável: `int`)
 - 3 - Declarar variável (Nome da variável: `idade`, Tipo da variável: `float`)
- **Saída Esperada:**
 - Declarada: `idade -> int`
 - Erro: `'idade'` já declarada neste escopo.

```

=== MENU ===
1 - Adicionar escopo
... 2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3

Nome da variável: idade

Tipo da variável: int
Declarada: idade -> int

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3

Nome da variável: idade

Tipo da variável: float
Erro: 'idade' já declarada neste escopo.

```

Figura 6: Evidência do Caso de Teste 6: Bloqueio de redeclaração no mesmo escopo.

3.7 Caso de Teste 7 – Proteção do Escopo Global

- **Objetivo:** Verificar se o sistema impede a remoção do escopo global, garantindo que sempre exista pelo menos um escopo ativo na tabela de símbolos.
- **Entrada:**
 - 2 - Remover escopo
- **Saída Esperada:**

Erro: o escopo global não pode ser removido.

A execução demonstrada na Figura 7 evidencia que, ao tentar remover o único escopo existente (escopo global), o sistema bloqueia a operação e informa o usuário por meio de uma mensagem de erro apropriada. Esse comportamento garante a integridade da estrutura de gerenciamento de escopos,

uma vez que todo programa deve possuir ao menos um escopo ativo para armazenar símbolos.

```
Nome da variável: idade

Tipos disponíveis:
int
float
string
char
bool

Tipo da variável: float
Declarada: idade -> float

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 2
Erro: Não é possível remover o Escopo Global.
```

Figura 7: Evidência do Caso de Teste 7: Proteção contra remoção do escopo global.

3.8 Caso de Teste 8 – Visualização da Tabela de Símbolos

- **Objetivo:** Verificar se o sistema exibe corretamente a estrutura da pilha de escopos e suas respectivas tabelas hash.

- **Entrada:**

– 5 - Mostrar tabela

- **Saída Esperada:**

Escopos atuais:

Escopo 0: { }

Escopo 1: {'x': 'int', 'nome': 'string'}

Escopo 2: {'x': 'float', 'idade': 'int'}

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 5

Escopos atuais:
Escopo 0: {}
Escopo 1: {'x': 'int', 'nome': 'string'}
Escopo 2: {'x': 'float', 'idade': 'int'}
```

Figura 8: Evidência do Caso de Teste 8: Estrutura da pilha de escopos exibida em terminal.

Referências

- [1] Alfred V. AHO et al. *Compilers: Principles, Techniques, and Tools*. 2^a ed. Boston: Addison-Wesley, 2006.
- [2] André Luiz Villar FORBELLONE e Henri Frederico EBERSPÄCHER. *Lógica de Programação: A Construção de Algoritmos e Estruturas de Dados*. 3^a ed. São Paulo: Pearson, 2005.
- [3] Python Software Foundation. *Python Documentation*. Acesso em: 9 jun. 2026. 2026. URL: <https://python.org>.
- [4] Thomas H. CORMEN et al. *Algoritmos: Teoria e Prática*. 3^a ed. Rio de Janeiro: Elsevier, 2012.